



From MOCK to REAL — every integration, mapped.



From MOCK to REAL — a checklist of every mock in the codebase, where it lives, and exactly what to change.

0. Quick map

#	Integration	Currently	File(s) to edit	Env var(s) you need
1	Google Places API (leads)	Mock generator	<code>lib/mock-leads.js</code> , <code>app/api/[...path]/route.js</code>	<code>GOOGLE_PLACES_API_KEY</code>
2	Google OAuth login	Mock (creates fake Google user)	<code>app/api/[...path]/route.js</code> , <code>app/login/page.js</code> , <code>app/register/page.js</code>	<code>GOOGLE_CLIENT_ID</code> , <code>GOOGLE_CLIENT_SECRET</code>
3	GitHub repo + Vercel deploy	Mock (fake URLs, marks ready after 2.5s)	<code>app/api/[...path]/route.js</code> (<code>/deployments</code> POST)	<code>GITHUB_TOKEN</code> , <code>GITHUB_OWNER</code> , <code>VERCEL_TOKEN</code>
4	WhatsApp Business API	Mock (stores queued row)	<code>app/api/[...path]/route.js</code> (<code>/whatsapp/send</code>)	<code>WHATSAPP_TOKEN</code> , <code>WHATSAPP_PHONE_NUMBER_ID</code>
5	Stats 14-day trend	Mock sinusoid	<code>app/api/[...path]/route.js</code> (<code>/stats</code> handler)	—
6	Gemini 2.5 Flash	Already REAL via Emergent proxy	<code>lib/gemini.js</code>	<code>EMERGENT_LLM_KEY</code> (already set)
7	Database	Already REAL — MongoDB	<code>.env</code>	<code>MONGO_URL</code> , <code>DB_NAME</code>

Everything else (register, login, JWT, CRM, dashboard, demo generator UI, demo preview) is already real.

1. Google Places API — real leads

1.1 Get the key

- URL: <https://console.cloud.google.com/apis/credentials>
- Create project → enable "Places API (New)" → Create credentials → API Key
- Restrict the key by referrer/IP for safety

1.2 Add to env

File: `/app/.env`

```
GOOGLE_PLACES_API_KEY=AIzaSy...
```

Then restart: `sudo supervisorctl restart nextjs`

1.3 What to change

Delete the `generateLeads(...)` call chain and replace with a real API call.

File: `/app/app/api/[...path]/route.js`

Find this block (around the `/leads/search` route):

```
if (route === '/leads/search' && method === 'POST') {  
  const b = await request.json()  
  if (!b.city || !b.category) return err('city and category required')  
  const leads = generateLeads(b.city, b.category, b.limit || 12)  
  ...  
}
```

Replace with:

```

if (route === '/leads/search' && method === 'POST') {
  const b = await request.json()
  if (!b.city || !b.category) return err('city and category required')

  const query = `${b.category} in ${b.city}`
  const gRes = await fetch('https://places.googleapis.com/v1/places:searchText', {
    method: 'POST',
    headers: {
      'Content-Type': 'application/json',
      'X-Goog-API-Key': process.env.GOOGLE_PLACES_API_KEY,
      // Only ask for the fields you need to reduce cost
      'X-Goog-FieldMask': 'places.id,places.displayName,places.formattedAddress,places.rating,places.userRatingCount',
    },
    body: JSON.stringify({ textQuery: query, languageCode: 'en', pageSize: b.limit || 12 })
  })
  if (!gRes.ok) return err('Google Places call failed: ' + await gRes.text(), 502)
  const gData = await gRes.json()

  const leads = (gData.places || []).map(p => ({
    id: p.id,
    name: p.displayName?.text || 'Unknown',
    category: b.category,
    city: b.city,
    address: p.formattedAddress || '',
    phone: p.nationalPhoneNumber || null,
    email: null, // Places API does not return email
    website: p.websiteUri || null,
    rating: p.rating || 0,
    reviewCount: p.userRatingCount || 0,
    hasInstagram: false, // enrich via a separate step if needed
    hasFacebook: false,
    priceLevel: p.priceLevel || 0,
    lat: p.location?.latitude,
    lng: p.location?.longitude,
    opportunityScore: null,
  }))

  let filtered = leads
  if (b.filters?.noWebsite) filtered = filtered.filter(l => !l.website)
  if (b.filters?.minRating) filtered = filtered.filter(l => l.rating >= b.filters.minRating)
  return ok({ leads: filtered, count: filtered.length, source: 'google_places' })
}

```

Optional: Also delete `/app/lib/mock-leads.js` OR keep it for offline testing. If you delete it, remove its import line at the top of `route.js` :

```
import { generateLeads } from '@lib/mock-leads' // <-- delete this line
```

And keep the `CATEGORIES` list somewhere (move to a new file or inline in `/app/app/dashboard/leads/page.js`).

1.4 Test

```
curl -X POST http://localhost:3000/api/leads/search \
  -H "Content-Type: application/json" \
  -d '{"city":"Austin","category":"restaurant","limit":5}'
```

2. Google OAuth — real login

2.1 Get credentials

- URL: <https://console.cloud.google.com/apis/credentials>
- Create OAuth 2.0 Client ID (type: Web application)
- Authorized JavaScript origins: `https://YOUR_DOMAIN` and `http://localhost:3000`
- Authorized redirect URIs: `https://YOUR_DOMAIN/api/auth/callback/google`

2.2 Env

```
GOOGLE_CLIENT_ID=xxx.apps.googleusercontent.com
GOOGLE_CLIENT_SECRET=xxx
NEXTAUTH_URL=https://YOUR_DOMAIN
```

2.3 Recommended path — use NextAuth (cleanest)

Install:

```
cd /app && yarn add next-auth
```

Delete the mock `/auth/google` endpoint and the mock button handlers.

Add `/app/app/api/auth/[...nextauth]/route.js`:

```
import NextAuth from 'next-auth'
import GoogleProvider from 'next-auth/providers/google'

const handler = NextAuth({
  providers: [
    GoogleProvider({
      clientId: process.env.GOOGLE_CLIENT_ID,
      clientSecret: process.env.GOOGLE_CLIENT_SECRET,
    })
  ],
  callbacks: {
    async signIn({ user, account }) {
      // Upsert into your Mongo `users` collection here
      return true
    },
    async jwt({ token, user }) { if (user) token.uid = user.id; return token },
    async session({ session, token }) { session.user.id = token.uid; return session },
  }
})
export { handler as GET, handler as POST }
```

Update `/app/app/login/page.js` and `/app/app/register/page.js` — replace the `googleLogin` function body with:

```
import { signIn } from 'next-auth/react'
// ...
const googleLogin = () => signIn('google', { callbackUrl: '/dashboard' })
```

Wrap the app in `<SessionProvider>` (edit `/app/app/providers.js`):

```
'use client'
import { SessionProvider } from 'next-auth/react'
import { ThemeProvider } from 'next-themes'

export function Providers({ children }) {
  return (
    <SessionProvider>
      <ThemeProvider attribute="class" defaultTheme="dark">
        {children}
      </ThemeProvider>
    </SessionProvider>
  )
}
```

2.4 Alternative — keep the manual flow

If you don't want NextAuth, do this in `/app/app/api/[...path]/route.js` in the `/auth/google` handler:

```

if (route === '/auth/google' && method === 'POST') {
  const b = await request.json() // expects { credential } from Google Identity Services on frontend
  const gRes = await fetch(`https://oauth2.googleapis.com/tokeninfo?id_token=${b.credential}`)
  if (!gRes.ok) return err('Invalid Google token', 401)
  const info = await gRes.json()
  if (info.aud !== process.env.GOOGLE_CLIENT_ID) return err('Bad audience', 401)
  const email = info.email.toLowerCase()
  let user = await db.collection('users').findOne({ email })
  if (!user) {
    user = { id: uuidv4(), name: info.name, email, role: 'sales', provider: 'google', createdAt: new Date() }
    await db.collection('users').insertOne(user)
  }
  const token = signToken(user)
  return ok({ token, user: { id: user.id, name: user.name, email: user.email, role: user.role } })
}

```

Then on the frontend add Google Identity Services and pass the `credential` to `/api/auth/google`.

3. GitHub + Vercel — real one-click deploy

This one is the biggest swap. Currently, `/deployments` POST just stores fake URLs.

3.1 Get credentials

- GitHub PAT: <https://github.com/settings/tokens?type=beta> — Fine-grained token, permissions: `repo` (Read + Write), `workflows`. Optional: use an org account.
- Vercel Token: <https://vercel.com/account/tokens> — Create → scope: your team/personal.

3.2 Env

```

GITHUB_TOKEN=github_pat_xxx
GITHUB_OWNER=your-github-username-or-org # where repos will be created
VERCEL_TOKEN=xxx
VERCEL_TEAM_ID=team_xxx # optional, if the repo should go into a team

```

3.3 What to change

File: `/app/app/api/[...path]/route.js` — the `/deployments` POST handler.

Currently:

```
if (route === '/deployments' && method === 'POST') {
  const t = verifyToken(request); if (!t) return err('Unauthorized', 401)
  const b = await request.json()
  const slug = (b.business?.name || 'demo').toLowerCase().replace(/^[a-z0-9]+/g, '-').replace(/^-|-$/, '')
  const doc = { ... status: 'building', repoUrl: `...`, liveUrl: `...` }
  await db.collection('deployments').insertOne(doc)
  setTimeout(async () => { ... }, 2500) // <-- mock
  return ok(strip(doc))
}
```

Replace the mock section with:

```

if (route === '/deployments' && method === 'POST') {
  const t = verifyToken(request); if (!t) return err('Unauthorized', 401)
  const b = await request.json()
  const slug = (b.business?.name || 'demo').toLowerCase().replace(/[a-z0-9]+/g, '-').replace(/^-|-$/g, '')
  const repoName = `demo-${slug}-${Date.now().toString(36)}`

  // 1) Fetch the demo we already generated
  const demoRow = await db.collection('demos').findOne({ id: b.demoId })
  if (!demoRow) return err('Demo not found', 404)

  // 2) Build the site HTML string (you can render your DemoPreview.js to string
  // with react-dom/server OR just template a static index.html with the JSON payload)
  const html = buildStaticSiteHtml(demoRow.demo, demoRow.business) // implement this helper

  // 3) Create a GitHub repo
  const ghCreate = await fetch('https://api.github.com/user/repos', {
    method: 'POST',
    headers: {
      'Authorization': `Bearer ${process.env.GITHUB_TOKEN}`,
      'Accept': 'application/vnd.github+json',
    },
    body: JSON.stringify({ name: repoName, private: false, auto_init: true })
  })
  if (!ghCreate.ok) return err('GitHub repo create failed: ' + await ghCreate.text(), 502)
  const repo = await ghCreate.json()

  // 4) Commit index.html
  const b64 = Buffer.from(html).toString('base64')
  const ghPut = await fetch(`https://api.github.com/repos/${repo.full_name}/contents/index.html`, {
    method: 'PUT',
    headers: { 'Authorization': `Bearer ${process.env.GITHUB_TOKEN}`, 'Accept': 'application/vnd.github+json' },
    body: JSON.stringify({ message: 'initial demo site', content: b64 })
  })
  if (!ghPut.ok) return err('GitHub commit failed: ' + await ghPut.text(), 502)

  // 5) Trigger Vercel deploy
  const vRes = await fetch(`https://api.vercel.com/v13/deployments${process.env.VERCEL_TEAM_ID ? `?teamId=${process.env.VERCEL_TEAM_ID}` : ''}`, {
    method: 'POST',
    headers: { 'Authorization': `Bearer ${process.env.VERCEL_TOKEN}`, 'Content-Type': 'application/json' },
    body: JSON.stringify({
      name: repoName,
      gitSource: { type: 'github', repo: repo.full_name, ref: 'main' },
      target: 'production',
      projectSettings: { framework: null }, // static site
    })
  })
  if (!vRes.ok) return err('Vercel deploy failed: ' + await vRes.text(), 502)
  const dep = await vRes.json()

  const doc = {

```

```

    id: uuidv4(),
    userId: t.id,
    demoId: b.demoId,
    business: b.business,
    status: dep.readyState === 'READY' ? 'ready' : 'building',
    repoUrl: repo.html_url,
    liveUrl: `https://${dep.url}`, // Vercel returns the URL without protocol
    vercelDeploymentId: dep.id,
    createdAt: new Date(),
  }
  await db.collection('deployments').insertOne(doc)
  return ok(strip(doc))
}

```

Add a helper somewhere (e.g. `/app/lib/site-builder.js`):

```

export function buildStaticSiteHtml(demo, business) {
  return `<!doctype html><html><head><meta charset="utf-8"><title>${business.name}</title>...
  <!-- inline your Tailwind CDN + the same JSX-like layout as DemoPreview.js -->
  </body></html>`
}

```

(The cleanest approach is to use `renderToStaticMarkup` from `react-dom/server` on the `<DemoPreview />` component and inline the resulting HTML.)

3.4 Poll status (optional)

Add a new GET route:

```

if (route.startsWith('/deployments/') && method === 'GET') {
  const id = route.split('/')[2]
  const row = await db.collection('deployments').findOne({ id })
  if (!row?.vercelDeploymentId) return ok(strip(row))
  const s = await fetch(`https://api.vercel.com/v13/deployments/${row.vercelDeploymentId}`, {
    headers: { 'Authorization': `Bearer ${process.env.VERCEL_TOKEN}` }
  }).then(r => r.json())
  const status = s.readyState === 'READY' ? 'ready' : 'building'
  await db.collection('deployments').updateOne({ id }, { $set: { status } })
  return ok(strip({ ...row, status }))
}

```

The frontend `/app/app/dashboard/deployments/page.js` already polls every 3s — it will just work.

4. WhatsApp Business API — real outreach

4.1 Get credentials

- URL: <https://developers.facebook.com/apps>
- Create App → Add WhatsApp product → Get "Temporary access token" (24h) OR create a System User for permanent token.
- Get your Phone Number ID from the WhatsApp > API Setup page.
- Approve at least one message template (e.g. `agency_intro` with 1 variable).

4.2 Env

```
WHATSAPP_TOKEN=EAA...  
WHATSAPP_PHONE_NUMBER_ID=123456789012345
```

4.3 What to change

File: `/app/app/api/[...path]/route.js` — the `/whatsapp/send` handler.

Currently mocks. Replace with:

```

if (route === '/whatsapp/send' && method === 'POST') {
  const t = verifyToken(request); if (!t) return err('Unauthorized', 401)
  const b = await request.json()
  // Send an approved template message
  const waRes = await fetch(`https://graph.facebook.com/v20.0/${process.env.WHATSAPP_PHONE_NUMBER_ID}/messages`,
    method: 'POST',
    headers: {
      'Authorization': `Bearer ${process.env.WHATSAPP_TOKEN}`,
      'Content-Type': 'application/json',
    },
    body: JSON.stringify({
      messaging_product: 'whatsapp',
      to: b.to, // '+15551234567'
      type: 'template',
      template: {
        name: b.template || 'agency_intro',
        language: { code: 'en_US' },
        components: [{ type: 'body', parameters: (b.variables ? Object.values(b.variables) : []).map(v => ({ type
      })
    })
  })
  if (!waRes.ok) return err('WhatsApp send failed: ' + await waRes.text(), 502)
  const waData = await waRes.json()
  const doc = {
    id: uuidv4(),
    userId: t.id,
    to: b.to,
    template: b.template,
    variables: b.variables || {},
    status: 'sent',
    whatsappMessageId: waData.messages?.[0]?.id,
    createdAt: new Date(),
  }
  await db.collection('whatsapp_messages').insertOne(doc)
  return ok(strip(doc))
}

```

4.4 Rules

- You can only send free-form messages within a 24h window after the prospect messages you first. Cold outreach must use a pre-approved template.
- Set up a webhook at `/api/whatsapp/webhook` to catch inbound replies.

5. Stats trend — real data

File: `/app/app/api/[...path]/route.js` — the `/stats` handler.

Currently uses a mock sinusoid. Replace with a real aggregation:

```
if (route === '/stats' && method === 'GET') {
  const t = verifyToken(request); if (!t) return err('Unauthorized', 401)

  const [leadsSaved, demos, deployments, hotLeads] = await Promise.all([
    db.collection('saved_leads').countDocuments({ userId: t.id }),
    db.collection('demos').countDocuments({}),
    db.collection('deployments').countDocuments({ userId: t.id }),
    db.collection('saved_leads').countDocuments({ userId: t.id, 'score.score': { $gte: 70 } }),
  ])

  // 14-day real trend
  const since = new Date(Date.now() - 14 * 24 * 60 * 60 * 1000)
  const trendRaw = await db.collection('saved_leads').aggregate([
    { $match: { userId: t.id, createdAt: { $gte: since } } },
    { $group: {
      _id: { $dateToString: { format: '%Y-%m-%d', date: '$createdAt' } },
      leads: { $sum: 1 },
    } },
    { $sort: { _id: 1 } }
  ]).toArray()

  // Fill missing days with 0
  const days = []
  for (let i = 13; i >= 0; i--) {
    const d = new Date(Date.now() - i * 864e5).toISOString().slice(0, 10)
    const found = trendRaw.find(r => r._id === d)
    days.push({ day: d, leads: found?.leads || 0, messages: 0 })
  }
  return ok({ leadsSaved, demos, deployments, hotLeads, trend: days })
}
```

You can join `whatsapp_messages` for the `messages` field similarly.

6. Gemini (already REAL)

Already live via Emergent's LLM proxy in `/app/lib/gemini.js`. Model: `gemini/gemini-2.5-flash`. Universal `EMERGENT_LLM_KEY` in `.env`.

If you want to use your own Google Gemini API key later:

- URL: <https://aistudio.google.com/apikey>

- Env: `GEMINI_API_KEY=AIZA...`
- Replace `/app/lib/gemini.js` with:

```
export async function callGemini(messages, { jsonMode = true, temperature = 0.7 } = {}) {
  const key = process.env.GEMINI_API_KEY
  const url = `https://generativelanguage.googleapis.com/v1beta/models/gemini-2.5-flash:generateContent?key=${key}`
  const contents = messages.map(m => ({ role: m.role === 'assistant' ? 'model' : 'user', parts: [{ text: m.content }]}))
  const res = await fetch(url, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({
      contents,
      generationConfig: { temperature, responseMimeType: jsonMode ? 'application/json' : 'text/plain' }
    })
  })
  if (!res.ok) throw new Error(`Gemini failed ${res.status}: ${await res.text()}`)
  const data = await res.json()
  const text = data.candidates?.[0]?.content?.parts?.[0]?.text || ''
  return jsonMode ? JSON.parse(text) : text
}
```

7. Database — using your own

Currently uses `mongodb://localhost:27017` with DB name `agencyos_ai`.

File: `/app/.env`

```
MONGO_URL=mongodb+srv://user:pass@your-cluster.mongodb.net
DB_NAME=agencyos_ai
```

Then restart. No code changes needed — `/app/app/api/[...path]/route.js` reads these vars.

If switching to Postgres/Supabase/MySQL: you'll have to rewrite the DB calls in `route.js` (all use `db.collection('...').findOne / insertOne / updateOne / aggregate`). Consider using Prisma to make this less painful.

8. Full env file — final version

`/app/.env` :

```
# Database
MONGO_URL=mongodb+srv://...
DB_NAME=agencyos_ai

# App
NEXT_PUBLIC_BASE_URL=https://YOUR_DOMAIN
CORS_ORIGINS=*
JWT_SECRET=<generate a long random string>

# LLM (keep Emergent OR swap to your own Gemini key – see section 6)
EMERGENT_LLM_KEY=sk-emergent-...
# GEMINI_API_KEY=AIza...

# Google Places (section 1)
GOOGLE_PLACES_API_KEY=

# Google OAuth (section 2)
GOOGLE_CLIENT_ID=
GOOGLE_CLIENT_SECRET=
NEXTAUTH_URL=https://YOUR_DOMAIN
NEXTAUTH_SECRET=<random>

# GitHub + Vercel (section 3)
GITHUB_TOKEN=
GITHUB_OWNER=
VERCEL_TOKEN=
VERCEL_TEAM_ID=

# WhatsApp (section 4)
WHATSAPP_TOKEN=
WHATSAPP_PHONE_NUMBER_ID=
```

9. Search-and-replace cheat sheet

Find	Replace with	Where
<code>generateLeads(b.city, b.category, ...)</code>	Google Places <code>searchText</code> call	<code>route.js</code>
<code>if (route === '/auth/google' block</code>	Real token verification OR NextAuth	<code>route.js</code>
<code>setTimeout(async () => { ... status: 'ready' ... }, 2500)</code>	GitHub repo create + Vercel deploy	<code>route.js</code>
<code>if (route === '/whatsapp/send' mock</code>	Graph API POST	<code>route.js</code>
<code>const days = Array.from({ length: 14 }, ...)</code>	Mongo aggregation	<code>route.js</code> /stats
<code>MOCK / TODO / GET YOUR ... HERE</code> comments	Delete after swap	search entire <code>route.js</code>

Run this to find every mock marker:

```
grep -rn "MOCK\|TODO\|GET YOUR" /app/app /app/lib
```

10. Deployment checklist

Before going live:

- ☐ Rotate `JWT_SECRET` to a long random string
- ☐ Rotate `EMERGENT_LLM_KEY` OR swap to your own `GEMINI_API_KEY`
- ☐ Set `NEXT_PUBLIC_BASE_URL` to your production domain
- ☐ Enable CORS restriction (`CORS_ORIGINS=https://yourdomain.com`)
- ☐ Add rate limiting middleware (e.g. `express-rate-limit` or Upstash Redis)
- ☐ Delete all `MOCK` comments after each swap
- ☐ Add real error monitoring (Sentry)
- ☐ Set `NODE_ENV=production` in the production env

